

LAB08: Stima dei parametri, likelihood e simulazione

Antonio Scala

Obiettivi

In questo laboratorio studiamo la stima dei parametri nei modelli stocastici, percorrendo la direzione inversa rispetto alla simulazione.

Nei laboratori precedenti abbiamo spesso fissato i parametri e generato traiettorie. Qui facciamo il contrario:

dati osservati $\longrightarrow$ stima dei parametri.

L'obiettivo non è solo applicare formule note, ma capire quale strategia inferenziale usare in situazioni diverse:

1. quando la likelihood è esplicita;
2. quando la likelihood deve essere costruita sulla traiettoria;
3. quando la likelihood non è disponibile in forma chiusa, ma il modello è simulabile.

Alla fine del laboratorio dovrete essere in grado di:

1. scrivere e massimizzare una log-likelihood semplice;
2. confrontare una MLE analitica con una stima numerica;
3. interpretare la curvatura della log-likelihood;
4. costruire la likelihood di una traiettoria per un processo di salto;
5. stimare tassi come rapporto tra numero di eventi e tempo di esposizione;
6. costruire una likelihood approssimata per una SDE osservata a tempi discreti;
7. usare simulazioni del modello per stimare parametri quando la likelihood è intrattabile;
8. distinguere stima dei parametri e diagnostica del modello.

Il filo conduttore

La stima di un parametro non è una singola tecnica. Dipende da quanta struttura probabilistica conosciamo.

In questo laboratorio attraversiamo tre livelli.

Livello 1 – Likelihood esplicita

Se conosciamo la densità dei dati,

$$p(x \mid \theta),$$

possiamo scrivere

$$L(\theta) = \prod_i p(x_i \mid \theta)$$

e massimizzare

$$\ell(\theta) = \log L(\theta).$$

Esempio: tempi di attesa esponenziali di un processo di Poisson.

Livello 2 – Likelihood di traiettoria

Se i dati sono una traiettoria dinamica, non possiamo trattare le osservazioni come indipendenti. Dobbiamo scrivere la probabilità dell'intera traiettoria.

Esempio: processo di salto a due stati.

Livello 3 – Modelli simulabili ma likelihood intrattabile

In molti modelli complessi possiamo simulare traiettorie, ma non sappiamo calcolare esplicitamente la probabilità dei dati osservati. In questo caso confrontiamo statistiche osservate e statistiche simulate.

Esempio: modello SIR stocastico osservato solo tramite statistiche aggregate.

Il messaggio centrale è:

quando la likelihood è disponibile, la usiamo; quando non lo è, la simulazione diventa parte dell'inferenza.

Organizzazione dei file

La cartella del laboratorio può essere organizzata così:

```
Lab08/
|-- data/
|   |-- poisson_interarrivals.csv
|   |-- jump_process.csv
|   |-- ou_process.csv
|   |-- sir_observed_stats.csv
|   |-- soluzioni_parametri.csv      # solo per il docente
|-- likelihoods.py
|-- simulate_models.py
|-- fit_models.py
|-- diagnostics.py
|-- main.py
|-- requirements.txt
`-- output/
```

Il file `soluzioni_parametri.csv` contiene i parametri usati per generare i dati sintetici. È una chiave per il docente e non va usato durante l'analisi.

Dipendenze Python

Useremo solo librerie standard per calcolo scientifico, ottimizzazione e grafici:

```
numpy
pandas
matplotlib
scipy
```

Installazione:

```
pip install numpy pandas matplotlib scipy
```

oppure:

```
pip install -r requirements.txt
```

Parte A – Tempi di attesa di un processo di Poisson

A.1 Modello

Un processo di Poisson omogeneo con tasso λ produce eventi nel tempo.

Se osserviamo i tempi di interarrivo

$$s_1, s_2, \dots, s_n,$$

allora, nel caso omogeneo, questi tempi sono indipendenti e distribuiti esponenzialmente:

$$p(s \mid \lambda) = \lambda e^{-\lambda s}, \quad s \geq 0.$$

Il parametro λ ha dimensione di un inverso di tempo: rappresenta il numero medio di eventi per unità di tempo.

A.2 Generazione dei dati

Per iniziare, generiamo dati sintetici con parametro noto:

```
import numpy as np

rng = np.random.default_rng(123)

lambda_true = 2.5
n = 300

inter_arrivals = rng.exponential(scale=1/lambda_true, size=n)

Salvate i dati in un file:

import pandas as pd

df = pd.DataFrame({"s": inter_arrivals})
df.to_csv("data/poisson_interarrivals.csv", index=False)
```

A.3 Log-likelihood

Per un campione di tempi di attesa indipendenti,

$$L(\lambda) = \prod_{i=1}^n \lambda e^{-\lambda s_i}.$$

Quindi

$$L(\lambda) = \lambda^n \exp\left(-\lambda \sum_{i=1}^n s_i\right).$$

La log-likelihood è

$$\ell(\lambda) = n \log \lambda - \lambda \sum_{i=1}^n s_i.$$

Implementatela:

```
import numpy as np

def loglik_exponential_rate(lam, s):
    if lam <= 0:
        return -np.inf
    n = len(s)
    return n*np.log(lam) - lam*np.sum(s)
```

A.4 MLE in forma chiusa

Derivando:

$$\frac{d\ell}{d\lambda} = \frac{n}{\lambda} - \sum_{i=1}^n s_i.$$

La condizione del primo ordine dà

$$\hat{\lambda} = \frac{n}{\sum_i s_i} = \frac{1}{\bar{s}}.$$

Implementazione:

```
def mle_exponential_rate_closed(s):
    return len(s) / np.sum(s)
```

A.5 MLE numerica

Usiamo anche una stima numerica, per vedere come la massima verosimiglianza si traduce in un problema di ottimizzazione.

```
from scipy.optimize import minimize_scalar

res = minimize_scalar(
    lambda lam: -loglik_exponential_rate(lam, inter_arrivals),
    bounds=(0.01, 10.0),
    method="bounded"
)

lambda_hat_num = res.x
lambda_hat_closed = mle_exponential_rate_closed(inter_arrivals)
```

```

print("lambda vero:", lambda_true)
print("lambda MLE chiusa:", lambda_hat_closed)
print("lambda MLE numerica:", lambda_hat_num)

```

A.6 Forma della log-likelihood

Tracciate la log-likelihood in funzione di λ :

```

import matplotlib.pyplot as plt

grid = np.linspace(0.2, 6.0, 400)
ll_values = [loglik_exponential_rate(lam, inter_arrivals) for lam in grid]

plt.figure()
plt.plot(grid, ll_values)
plt.axvline(lambda_true, linestyle="--", label="vero")
plt.axvline(lambda_hat_closed, linestyle=":", label="MLE")
plt.xlabel(r"$\lambda$")
plt.ylabel(r"$\ell(\lambda)$")
plt.legend()
plt.tight_layout()
plt.savefig("output/poisson_loglik.png", dpi=150)
plt.show()

```

A.7 Curvatura e incertezza

Per questo modello,

$$\ell''(\lambda) = -\frac{n}{\lambda^2}.$$

Quindi, nel massimo,

$$-\ell''(\hat{\lambda}) = \frac{n}{\hat{\lambda}^2}.$$

L'approssimazione della varianza dello stimatore è

$$\widehat{\text{Var}}(\hat{\lambda}) \approx \frac{1}{-\ell''(\hat{\lambda})} = \frac{\hat{\lambda}^2}{n}.$$

Implementazione:

```

lambda_hat = lambda_hat_closed
se_lambda = lambda_hat / np.sqrt(n)

```

```
print("errore standard approssimato:", se_lambda)
```

A.8 Compiti

1. Ripetere la stima per $n = 30$, $n = 300$, $n = 3000$.
2. Disegnare la log-likelihood nei tre casi.
3. Osservare come cambia la curvatura attorno al massimo.
4. Ripetere la generazione dei dati con seed diversi e studiare la variabilità di $\hat{\lambda}$.
5. Confrontare la deviazione standard empirica delle stime con l'approssimazione $\hat{\lambda}/\sqrt{n}$.

A.9 Domande guida

1. La MLE numerica coincide con quella analitica?
 2. Perché la log-likelihood diventa più stretta quando aumenta n ?
 3. La curva è simmetrica attorno al massimo?
 4. Che cosa significa, in termini pratici, una log-likelihood molto piatta?
 5. Perché lavoriamo con la log-likelihood invece che con la likelihood?
-

Parte B – Processo di salto a due stati

B.1 Modello

Consideriamo un processo markoviano a tempo continuo con due stati:

$$1 \rightleftharpoons 2.$$

I tassi di transizione sono

$$w_{1 \rightarrow 2} = \alpha, \quad w_{2 \rightarrow 1} = \beta.$$

Se il sistema si trova nello stato 1, il tempo di attesa prima del salto verso 2 è esponenziale con tasso α .

Se il sistema si trova nello stato 2, il tempo di attesa prima del salto verso 1 è esponenziale con tasso β .

B.2 Simulazione della traiettoria

Implementiamo una simulazione esatta evento per evento.

```

import numpy as np
import pandas as pd

def simulate_two_state(alpha, beta, T, x0=1, seed=123):
    rng = np.random.default_rng(seed)

    t = 0.0
    x = x0

    times = [t]
    states = [x]
    jumps = []

    while t < T:
        if x == 1:
            rate = alpha
            new_x = 2
        else:
            rate = beta
            new_x = 1

        tau = rng.exponential(scale=1/rate)

        if t + tau > T:
            break

        t_next = t + tau
        jumps.append((t_next, x, new_x))

        t = t_next
        x = new_x

        times.append(t)
        states.append(x)

    return times, states, jumps

Generiamo una traiettoria:

alpha_true = 1.2
beta_true = 0.7
T = 100.0

times, states, jumps = simulate_two_state(alpha_true, beta_true, T)

df_jumps = pd.DataFrame(jumps, columns=["time", "from_state", "to_state"])
df_jumps.to_csv("data/jump_process.csv", index=False)

```


B.3 Statistiche sufficienti della traiettoria

Per stimare i tassi non serve conservare ogni dettaglio della traiettoria. Servono:

- N_{12} : numero di salti da 1 a 2;
- N_{21} : numero di salti da 2 a 1;
- T_1 : tempo totale passato nello stato 1;
- T_2 : tempo totale passato nello stato 2.

Calcoliamoli:

```
def two_state_sufficient_statistics(jumps, T, x0=1):
    N12 = 0
    N21 = 0
    T1 = 0.0
    T2 = 0.0

    t_prev = 0.0
    x = x0

    for t_jump, from_state, to_state in jumps:
        duration = t_jump - t_prev

        if x == 1:
            T1 += duration
        else:
            T2 += duration

        if from_state == 1 and to_state == 2:
            N12 += 1
        elif from_state == 2 and to_state == 1:
            N21 += 1

        x = to_state
        t_prev = t_jump

    duration = T - t_prev
    if x == 1:
        T1 += duration
    else:
        T2 += duration

    return N12, N21, T1, T2
```

B.4 Likelihood di traiettoria

La log-likelihood della traiettoria è

$$\ell(\alpha, \beta) = N_{12} \log \alpha + N_{21} \log \beta - \alpha T_1 - \beta T_2.$$

Questa formula ha una struttura intuitiva:

- ogni salto osservato contribuisce con il logaritmo del tasso corrispondente;
- ogni intervallo di permanenza contribuisce con un termine di sopravvivenza esponenziale.

Implementazione:

```
def loglik_two_state(alpha, beta, N12, N21, T1, T2):
    if alpha <= 0 or beta <= 0:
        return -np.inf
    return N12*np.log(alpha) + N21*np.log(beta) - alpha*T1 - beta*T2
```

B.5 MLE dei tassi

Derivando rispetto ad α e β :

$$\frac{\partial \ell}{\partial \alpha} = \frac{N_{12}}{\alpha} - T_1,$$

$$\frac{\partial \ell}{\partial \beta} = \frac{N_{21}}{\beta} - T_2.$$

Ponendo a zero:

$$\hat{\alpha} = \frac{N_{12}}{T_1}, \quad \hat{\beta} = \frac{N_{21}}{T_2}.$$

Implementazione:

```
N12, N21, T1, T2 = two_state_sufficient_statistics(jumps, T)
```

```
alpha_hat = N12 / T1
```

```
beta_hat = N21 / T2
```

```
print("N12, N21:", N12, N21)
```

```
print("T1, T2:", T1, T2)
```

```
print("alpha vero/stimato:", alpha_true, alpha_hat)
```

```
print("beta vero/stimato:", beta_true, beta_hat)
```

B.6 Diagnostica semplice

Simulate una nuova traiettoria usando i parametri stimati e confrontate:

1. numero totale di salti;
2. frazione di tempo passata nello stato 1;
3. frazione di tempo passata nello stato 2;
4. distribuzione dei tempi di permanenza.

La frazione stazionaria teorica nello stato 1 è

$$\pi_1 = \frac{\beta}{\alpha + \beta},$$

mentre nello stato 2 è

$$\pi_2 = \frac{\alpha}{\alpha + \beta}.$$

Confrontate queste quantità con

$$\hat{\pi}_1 = \frac{T_1}{T}, \quad \hat{\pi}_2 = \frac{T_2}{T}.$$

B.7 Compiti

1. Simulare una traiettoria con $T = 20$, $T = 100$, $T = 500$.
2. Stimare α e β nei tre casi.
3. Rappresentare graficamente la convergenza delle stime al crescere di T .
4. Ripetere più volte con seed diversi.
5. Studiare cosa succede quando uno dei due stati viene visitato poco.

B.8 Domande guida

1. Perché il denominatore della stima è il tempo totale passato nello stato di partenza?
 2. Perché non dividiamo per il tempo totale T ?
 3. Che cosa succede se T_1 è molto piccolo?
 4. In che senso T_i è un tempo di esposizione?
 5. Questa likelihood sarebbe corretta se osservassimo solo gli stati a tempi discreti e non i tempi di salto?
-

Parte C – Ornstein–Uhlenbeck osservato a tempi discreti

C.1 Modello

Consideriamo il processo di Ornstein–Uhlenbeck:

$$dX_t = -\gamma(X_t - \mu) dt + \sigma dW_t.$$

Il parametro $\gamma > 0$ controlla la velocità di ritorno verso la media μ .

Per semplicità, in questa parte assumiamo inizialmente che μ e σ siano noti e stimiamo solo γ .

C.2 Simulazione con Euler–Maruyama

```
import numpy as np

def simulate_ou(gamma, mu, sigma, x0, dt, T, seed=123):
    rng = np.random.default_rng(seed)
    n_steps = int(T/dt)

    x = np.empty(n_steps + 1)
    x[0] = x0

    for k in range(n_steps):
        dW = rng.normal(0.0, np.sqrt(dt))
        x[k+1] = x[k] - gamma*(x[k]-mu)*dt + sigma*dW

    t = np.linspace(0, T, n_steps + 1)
    return t, x
```

Generiamo dati:

```
gamma_true = 1.5
mu_true = 2.0
sigma_true = 0.8

dt = 0.01
T = 50.0

t, x = simulate_ou(gamma_true, mu_true, sigma_true, x0=2.0, dt=dt, T=T)

df_ou = pd.DataFrame({"t": t, "x": x})
df_ou.to_csv("data/ou_process.csv", index=False)
```

C.3 Likelihood approssimata di Euler–Maruyama

Per piccoli Δt ,

$$X_{k+1} \mid X_k = x_k \approx \mathcal{N}(x_k - \gamma(x_k - \mu)\Delta t, \sigma^2\Delta t).$$

Quindi la log-likelihood approssimata è

$$\ell(\gamma) = -\frac{1}{2} \sum_{k=0}^{N-1} \left[\log(2\pi\sigma^2\Delta t) + \frac{(x_{k+1} - x_k + \gamma(x_k - \mu)\Delta t)^2}{\sigma^2\Delta t} \right].$$

Implementazione:

```
def loglik_ou_euler_gamma(gamma, x, dt, mu, sigma):
    if gamma <= 0:
        return -np.inf

    xk = x[:-1]
    xnext = x[1:]

    mean = xk - gamma*(xk - mu)*dt
    var = sigma**2 * dt

    resid = xnext - mean

    ll = -0.5*np.sum(np.log(2*np.pi*var) + resid**2/var)
    return ll
```

C.4 Stima numerica

```
from scipy.optimize import minimize_scalar

res = minimize_scalar(
    lambda g: -loglik_ou_euler_gamma(g, x, dt, mu_true, sigma_true),
    bounds=(0.01, 5.0),
    method="bounded"
)

gamma_hat_num = res.x

print("gamma vero:", gamma_true)
print("gamma stimato numericamente:", gamma_hat_num)
```

C.5 Formula di regressione sugli incrementi

Definiamo

$$\Delta x_k = x_{k+1} - x_k.$$

Dal modello discretizzato,

$$\Delta x_k = -\gamma(x_k - \mu)\Delta t + \sigma\sqrt{\Delta t}\xi_k.$$

Quindi, se μ e σ sono noti, stimare γ equivale a una regressione degli incrementi Δx_k sulla variabile $(x_k - \mu)\Delta t$.

La MLE approssimata è

$$\hat{\gamma} = -\frac{\sum_k (x_k - \mu)\Delta x_k}{\Delta t \sum_k (x_k - \mu)^2}.$$

Implementazione:

```
def estimate_ou_gamma_regression(x, dt, mu):
    dx = x[1:] - x[:-1]
    y = x[:-1] - mu
    return -np.sum(y*dx) / (dt*np.sum(y**2))

gamma_hat_reg = estimate_ou_gamma_regression(x, dt, mu_true)

print("gamma stimato via regressione:", gamma_hat_reg)
```

C.6 Propagatore esatto

Per Ornstein-Uhlenbeck il propagatore esatto è noto ed è gaussiano:

$$X_{k+1} | X_k = x_k \sim \mathcal{N}\left(\mu + (x_k - \mu)e^{-\gamma\Delta t}, \frac{\sigma^2}{2\gamma}(1 - e^{-2\gamma\Delta t})\right).$$

Quindi, per questo modello, si può costruire una likelihood esatta per dati campionati a passo finito.

Implementazione:

```
def loglik_ou_exact_gamma(gamma, x, dt, mu, sigma):
    if gamma <= 0:
        return -np.inf

    xk = x[:-1]
    xnext = x[1:]

    a = np.exp(-gamma*dt)
    mean = mu + (xk - mu)*a
```

```

var = sigma**2/(2*gamma) * (1 - np.exp(-2*gamma*dt))

resid = xnext - mean

ll = -0.5*np.sum(np.log(2*np.pi*var) + resid**2/var)
return ll

```

Confrontate la stima Euler–Maruyama con quella ottenuta dal propagatore esatto:

```

res_exact = minimize_scalar(
    lambda g: -loglik_ou_exact_gamma(g, x, dt, mu_true, sigma_true),
    bounds=(0.01, 5.0),
    method="bounded"
)

gamma_hat_exact = res_exact.x

print("gamma vero:", gamma_true)
print("gamma Euler:", gamma_hat_num)
print("gamma esatto:", gamma_hat_exact)

```

C.7 Campionamento più rado

Per vedere quando l'approssimazione di Euler–Maruyama diventa meno accurata, sottocampionate la traiettoria:

```

stride = 10
x_sparse = x[::stride]
dt_sparse = dt * stride

```

Ripetete le due stime:

1. likelihood Euler–Maruyama con `dt_sparse`;
2. likelihood esatta con `dt_sparse`.

C.8 Compiti

1. Stimare γ con Euler–Maruyama su una traiettoria fitta.
2. Stimare γ con il propagatore esatto sulla stessa traiettoria.
3. Sottocampionare la traiettoria e ripetere.
4. Confrontare l'errore di stima nei diversi casi.
5. Ripetere per $T = 5$, $T = 50$, $T = 200$.

C.9 Domande guida

1. Perché la likelihood Euler–Maruyama è solo approssimata?
2. Quando il propagatore esatto è preferibile?

3. Che cosa cambia se Δt aumenta?
 4. Perché la stima del drift può essere letta come regressione sugli incrementi?
 5. Perché lavorare con $\Delta x_k / \Delta t$ può essere numericamente rumoroso?
-

Parte D – Modello simulabile senza likelihood esplicita: SIR osservato parzialmente

D.1 Motivazione

Finora abbiamo usato modelli per cui la likelihood era esplicita o approssimabile.

Ma in molti casi reali osserviamo solo una parte del processo. Per esempio, in un'epidemia potremmo non osservare ogni singolo evento di infezione e guarigione, ma solo alcune grandezze aggregate:

- numero finale di rimossi;
- picco degli infetti;
- tempo del picco;
- conteggi giornalieri rumorosi;
- pochi punti della traiettoria.

In questo caso la likelihood completa può essere difficile o impossibile da scrivere, perché bisognerebbe sommare su molte traiettorie latenti compatibili con le osservazioni.

L'idea diventa allora:

se non sappiamo calcolare la probabilità dei dati, possiamo simulare il modello e confrontare dati simulati e dati osservati.

D.2 Modello SIR stocastico

Consideriamo una popolazione chiusa di taglia

$$N = S + I + R.$$

Gli eventi possibili sono:

1. infezione:

$$(S, I, R) \rightarrow (S - 1, I + 1, R)$$

con tasso

$$a_1(S, I, R) = \beta \frac{SI}{N} ;$$

2. guarigione:

$$(S, I, R) \rightarrow (S, I - 1, R + 1)$$

con tasso

$$a_2(S, I, R) = \gamma I .$$

I parametri da stimare sono

$$\theta = (\beta, \gamma) .$$

D.3 Simulazione Gillespie del SIR

```
def simulate_sir_gillespie(beta, gamma, N, IO, R0, T_max, seed=None):
    rng = np.random.default_rng(seed)

    S = N - IO - R0
    I = IO
    R = R0
    t = 0.0

    times = [t]
    S_values = [S]
    I_values = [I]
    R_values = [R]

    while t < T_max and I > 0:
        a_inf = beta * S * I / N
        a_rec = gamma * I
        a0 = a_inf + a_rec

        if a0 <= 0:
            break

        tau = rng.exponential(scale=1/a0)

        if t + tau > T_max:
            break

        t = t + tau

        if rng.random() < a_inf / a0:
            S -= 1
            I += 1
        else:
            I -= 1
            R += 1

        times.append(t)
```

```

        S_values.append(S)
        I_values.append(I)
        R_values.append(R)

    return {
        "t": np.array(times),
        "S": np.array(S_values),
        "I": np.array(I_values),
        "R": np.array(R_values)
    }

```

D.4 Dati osservati sintetici

Generiamo un'epidemia osservata con parametri veri.

```

N = 500
I0 = 5
R0 = 0
T_max = 80.0

beta_true = 0.45
gamma_true = 0.18

obs_traj = simulate_sir_gillespie(
    beta_true, gamma_true, N, I0, R0, T_max, seed=123
)

```

Invece di usare tutta la traiettoria, immaginiamo di osservare solo alcune statistiche aggregate.

```

def sir_summary_stats(traj):
    I = traj["I"]
    R = traj["R"]
    t = traj["t"]

    final_size = R[-1]
    peak_I = np.max(I)
    time_peak = t[np.argmax(I)]

    return np.array([final_size, peak_I, time_peak], dtype=float)

obs_stats = sir_summary_stats(obs_traj)

print("Statistiche osservate:", obs_stats)

```

Queste statistiche sono:

$$m_{\text{obs}} = (R_{\infty}, I_{\text{max}}, t_{\text{max}}).$$

D.5 Metodo dei momenti simulati

Per ogni valore candidato (β, γ) :

1. simuliamo il modello molte volte;
2. calcoliamo le statistiche riassuntive;
3. facciamo la media delle statistiche simulate;
4. confrontiamo questa media con le statistiche osservate.

Definiamo

$$\bar{m}_{\text{sim}}(\beta, \gamma) = \frac{1}{M} \sum_{r=1}^M m_{\text{sim}}^{(r)}(\beta, \gamma).$$

La funzione obiettivo è

$$Q(\beta, \gamma) = [m_{\text{obs}} - \bar{m}_{\text{sim}}(\beta, \gamma)]^T W [m_{\text{obs}} - \bar{m}_{\text{sim}}(\beta, \gamma)].$$

Nel caso più semplice usiamo $W = I$, ma conviene normalizzare le statistiche per evitare che una componente domini le altre solo per scala numerica.

```
def smm_objective_sir(params, obs_stats, N, IO, RO, T_max,
                      n_sim=20, scale=None, seed=1234):
    beta, gamma = params

    if beta <= 0 or gamma <= 0:
        return 1e12

    rng = np.random.default_rng(seed)
    stats = []

    for r in range(n_sim):
        sim_seed = int(rng.integers(0, 2**32 - 1))
        traj = simulate_sir_gillespie(
            beta, gamma, N, IO, RO, T_max, seed=sim_seed
        )
        stats.append(sir_summary_stats(traj))

    mean_stats = np.mean(stats, axis=0)

    if scale is None:
        scale = np.maximum(np.abs(obs_stats), 1.0)
```

```
diff = (mean_stats - obs_stats) / scale
return float(np.sum(diff**2))
```

D.6 Ottimizzazione numerica

La funzione obiettivo è rumorosa, perché ogni valutazione contiene simulazioni casuali. Usiamo un metodo derivative-free, ad esempio Nelder–Mead.

```
from scipy.optimize import minimize

scale = np.maximum(np.abs(obs_stats), 1.0)

res = minimize(
    smm_objective_sir,
    x0=np.array([0.35, 0.15]),
    args=(obs_stats, N, IO, RO, T_max),
    method="Nelder-Mead",
    options={"maxiter": 80, "xatol": 0.02, "fatol": 0.02}
)

beta_hat, gamma_hat = res.x

print("beta vero/stimato:", beta_true, beta_hat)
print("gamma vero/stimato:", gamma_true, gamma_hat)
print("valore obiettivo:", res.fun)
```

D.7 Mappa della funzione obiettivo

Per visualizzare il problema, calcoliamo $Q(\beta, \gamma)$ su una griglia.

```
import matplotlib.pyplot as plt

beta_grid = np.linspace(0.20, 0.70, 11)
gamma_grid = np.linspace(0.08, 0.35, 11)

Z = np.zeros((len(gamma_grid), len(beta_grid)))

for i, gam in enumerate(gamma_grid):
    for j, bet in enumerate(beta_grid):
        Z[i, j] = smm_objective_sir(
            [bet, gam], obs_stats, N, IO, RO, T_max,
            n_sim=10, scale=scale, seed=1000 + 100*i + j
        )

plt.figure()
plt.imshow(
    Z,
```

```

        origin="lower",
        extent=[beta_grid[0], beta_grid[-1], gamma_grid[0], gamma_grid[-1]],
        aspect="auto"
    )
    plt.colorbar(label="obiettivo SMM")
    plt.scatter([beta_true], [gamma_true], marker="*", s=150, label="vero")
    plt.scatter([beta_hat], [gamma_hat], marker="+", s=150, label="stimato")
    plt.xlabel(r"$\beta$")
    plt.ylabel(r"$\gamma$")
    plt.legend()
    plt.tight_layout()
    plt.savefig("output/sir_smm_landscape.png", dpi=150)
    plt.show()

```

D.8 Diagnostica del modello stimato

Una volta stimati $\hat{\beta}$ e $\hat{\gamma}$, simulate molte epidemie dal modello stimato e confrontate le statistiche simulate con quelle osservate.

```

def simulate_many_sir_stats(beta, gamma, n_rep, N, IO, R0, T_max, seed=999):
    rng = np.random.default_rng(seed)
    stats = []

    for _ in range(n_rep):
        sim_seed = int(rng.integers(0, 2**32 - 1))
        traj = simulate_sir_gillespie(beta, gamma, N, IO, R0, T_max, seed=sim_seed)
        stats.append(sir_summary_stats(traj))

    return np.array(stats)

stats_hat = simulate_many_sir_stats(
    beta_hat, gamma_hat, 200, N, IO, R0, T_max
)

labels = ["R finale", "picco I", "tempo picco"]

for k in range(3):
    plt.figure()
    plt.hist(stats_hat[:, k], bins=25, alpha=0.7)
    plt.axvline(obs_stats[k], linestyle="--", label="osservato")
    plt.xlabel(labels[k])
    plt.ylabel("frequenza")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"output/sir_check_{k}.png", dpi=150)
    plt.show()

```

D.9 Compiti

1. Generare un'epidemia osservata con parametri noti.
2. Calcolare le statistiche osservate:

$$R_{\infty}, \quad I_{\max}, \quad t_{\max}.$$

3. Implementare la funzione obiettivo SMM.
4. Stimare β e γ .
5. Visualizzare la funzione obiettivo su una griglia.
6. Simulare dal modello stimato e confrontare le statistiche osservate con quelle simulate.
7. Ripetere aumentando e diminuendo `n_sim`.

D.10 Domande guida

1. Le stime sono vicine ai parametri veri?
2. Quale statistica sembra più informativa su β ?
3. Quale statistica sembra più informativa su γ ?
4. Cosa succede se si usa una sola simulazione per valutare la funzione obiettivo?
5. Il paesaggio della funzione obiettivo è liscio o rumoroso?
6. Esiste un minimo ben definito oppure una valle di parametri quasi equivalenti?
7. Perché questa procedura non è una massima likelihood?

Parte E – ABC rejection sampler facoltativo

Questa parte è facoltativa. Serve a mostrare un secondo modo di usare la simulazione quando la likelihood è intrattabile.

E.1 Idea

Nel quadro bayesiano vorremmo calcolare

$$p(\theta \mid y_{\text{obs}}) \propto p(y_{\text{obs}} \mid \theta)p(\theta).$$

Ma se la likelihood

$$p(y_{\text{obs}} \mid \theta)$$

non è calcolabile, possiamo usare simulazioni.

L'idea di ABC è:

1. estrarre un parametro dal prior;
2. simulare dati dal modello;
3. confrontare dati simulati e osservati tramite statistiche;
4. accettare il parametro se la simulazione è abbastanza vicina ai dati osservati.

E.2 Versione semplice

Per semplicità stimiamo solo β , fissando γ al valore vero o a un valore assunto noto.

```
def abc_rejection_sir_beta(obs_stats, gamma_fixed,
                          N, IO, RO, T_max,
                          n_accepted=200,
                          eps_abc=0.30,
                          seed=12345):
    rng = np.random.default_rng(seed)
    accepted = []
    n_trials = 0

    scale = np.maximum(np.abs(obs_stats), 1.0)

    while len(accepted) < n_accepted:
        beta_try = rng.uniform(0.10, 0.80)

        sim_seed = int(rng.integers(0, 2**32 - 1))
        traj = simulate_sir_gillespie(
            beta_try, gamma_fixed, N, IO, RO, T_max, seed=sim_seed
        )
        sim_stats = sir_summary_stats(traj)

        dist = np.sqrt(np.sum(((sim_stats - obs_stats)/scale)**2))

        if dist <= eps_abc:
            accepted.append(beta_try)

        n_trials += 1

    return np.array(accepted), len(accepted)/n_trials
```

Esecuzione:

```
posterior_beta, acc_rate = abc_rejection_sir_beta(
    obs_stats,
    gamma_fixed=gamma_true,
    N=N,
    IO=IO,
    RO=RO,
```

```

        T_max=T_max,
        n_accepted=300,
        eps_abc=0.30
    )

    print("tasso di accettazione:", acc_rate)
    print("media ABC beta:", np.mean(posterior_beta))
    print("beta vero:", beta_true)

    plt.figure()
    plt.hist(posterior_beta, bins=30, density=True)
    plt.axvline(beta_true, linestyle="--", label="vero")
    plt.xlabel(r"$\beta$")
    plt.ylabel("densità approssimata")
    plt.legend()
    plt.tight_layout()
    plt.savefig("output/sir_abc_beta.png", dpi=150)
    plt.show()

```

E.3 Effetto della soglia

Ripetete con

$$\varepsilon_{ABC} \in \{0.15, 0.30, 0.60\}.$$

Confrontate:

- distribuzione dei parametri accettati;
- tasso di accettazione;
- distanza media tra statistiche simulate e osservate.

E.4 Domande guida

1. Cosa succede se ε_{ABC} è molto piccolo?
2. Cosa succede se ε_{ABC} è molto grande?
3. Perché ABC produce una distribuzione di parametri e non una singola stima?
4. In che senso ABC dipende dalla scelta delle statistiche riassuntive?
5. Perché ABC può diventare molto costoso computazionalmente?

Confronto finale tra i metodi

Alla fine del laboratorio confrontate i quattro casi.

Parte	Modello	Dati osservati	Likelihood	Metodo
A	Poisson / esponenziale	tempi di attesa i.i.d.	esplicita	MLE analitica e numerica
B	processo di salto a due stati	traiettoria completa	esplicita di traiettoria	MLE da conteggi e tempi
C	Ornstein–Uhlenbeck	traiettoria discretizzata	approssimata o esatta	MLE numerica
D	SIR parzialmente osservato	statistiche aggregate	intrattabile	momenti simulati
E	SIR parzialmente osservato	statistiche aggregate	intrattabile	ABC

Report finale richiesto

Consegnate un breve report in Markdown o PDF con:

1. una descrizione dei modelli analizzati;
2. le log-likelihood usate nelle Parti A, B e C;
3. le stime ottenute;
4. i grafici principali:
 - log-likelihood del Poisson;
 - convergenza dei tassi del processo di salto;
 - confronto Euler/propagatore esatto per OU;
 - mappa della funzione obiettivo SMM per SIR;
5. una discussione sulla diagnostica del modello stimato;
6. una risposta alle domande guida più importanti.

Il report non deve essere lungo. Deve però mostrare chiaramente:

- cosa è stato stimato;
- con quale criterio;
- con quali dati;
- con quale incertezza o variabilità;
- con quali limiti.

Cosa dovrete aver capito alla fine

Al termine del laboratorio dovrete aver verificato che:

1. per dati indipendenti la log-likelihood è una somma di contributi elementari;
2. per una traiettoria dinamica la likelihood deve rispettare la struttura temporale del processo;
3. nei processi di salto i tassi si stimano come numero di eventi diviso tempo di esposizione;
4. nelle SDE discretizzate la likelihood sugli incrementi è spesso solo approssimata;
5. quando il propagatore esatto è noto, può essere preferibile all'approssimazione di Euler-Maruyama;
6. quando la likelihood è intrattabile, la simulazione permette comunque di stimare parametri confrontando statistiche osservate e simulate;
7. la scelta delle statistiche riassuntive è cruciale nei metodi simulation-based;
8. una stima numerica non basta: bisogna sempre confrontare il modello stimato con i dati osservati.

Esercizi aggiuntivi

Esercizio 1 – Curvatura della log-likelihood esponenziale

Per la Parte A, calcolare analiticamente

$$-\ell''(\hat{\lambda})$$

e verificare numericamente che l'approssimazione quadratica della log-likelihood sia buona vicino al massimo.

Esercizio 2 – Processo di salto con pochi eventi

Simulare il processo a due stati con T molto piccolo. Studiare il caso in cui $N_{12} = 0$ oppure $N_{21} = 0$.

1. Cosa restituisce la formula della MLE?
2. Come interpretate una stima nulla?
3. Che ruolo avrebbe un prior bayesiano in questo caso?

Esercizio 3 – OU con μ ignoto

Modificare la Parte C stimando contemporaneamente γ e μ , assumendo σ nota.

Scrivere la likelihood approssimata e usare `scipy.optimize.minimize`.

Esercizio 4 – Scelta delle statistiche SIR

Nella Parte D sostituire le statistiche

$$(R_{\infty}, I_{\max}, t_{\max})$$

con valori di $I(t)$ osservati su una griglia di tempi.

Confrontare la qualità della stima.

Esercizio 5 – ABC su due parametri

Estendere ABC stimando contemporaneamente β e γ .

1. Scegliere prior uniformi ragionevoli.
2. Definire una distanza tra statistiche.
3. Studiare il tasso di accettazione.
4. Visualizzare il cloud dei parametri accettati nel piano (β, γ) .